

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA DE SOFTWARE



INTELIGENCIA ARTIFICIAL

SECCIÓN 2

Juegos con Poda Alfa-Beta

Aplicación al juego Conecta 4

GRUPO 03

INTEGRANTES:

Bautista De la Cruz Claudia Daniela

Carrascal Castro Priscila Maria

Ccahuana Quiñones Judith Valeria

Gil Sixi Alberto Luis

Medrano Ayma Nikol Arlet

Rosales Trinidad Jeanmarco Miguel

Saire Tello Fernando José

DOCENTE

Prof. Juan Gamarra Moreno

Lima, Perú – 2026

ÍNDICE

1. Introducción	4
2. Descripción y modelado del juego	4
2.1. Descripción del juego y reglas	4
2.2. Estados y jugadas legales	5
2.3. Función terminal y utilidad	6
2.4. Función de evaluación heurística	6
3. Diseño de la solución y algoritmos	7
3.1. Minimax	7
3.2. Poda Alfa-Beta	8
3.3. Modalidad de juego	8
4. Implementación	8
4.1. Estructura del código	8
4.2. Fragmentos comentados	9
5. Resultados y análisis	9
5.1. Partidas de prueba	9
5.2. Eficiencia de la poda	10
5.3. Análisis de resultados	10
6. Conclusiones	10
7. Anexo de prompts	10
8. Referencias	10

1. Introducción

El presente informe describe el diseño e implementación de un agente para el juego Conecta 4 que enfrenta a un jugador humano contra la máquina mediante el algoritmo Minimax con poda Alfa-Beta. Conecta 4 es un juego adversarial de dos jugadores, de suma cero e información perfecta, cuyo amplio árbol de búsqueda lo convierte en un caso adecuado para evaluar la eficiencia de esta técnica de optimización.

Debido al elevado costo computacional que implica explorar todas las posibles secuencias de juego, el agente utiliza una profundidad de búsqueda limitada a seis niveles y una función de evaluación heurística para estimar la conveniencia de los estados no terminales. Esta heurística considera la ocupación de la columna central, la evaluación de alineaciones parciales y el bloqueo de amenazas del oponente.

A lo largo del informe se presenta el modelado formal del juego, el diseño e implementación de los algoritmos utilizados y el análisis de los resultados obtenidos. Asimismo, se compara el desempeño de Minimax con y sin poda Alfa-Beta, evaluando la cantidad de nodos explorados y verificando que ambos algoritmos producen la misma decisión de juego.

2. Descripción y modelado del juego

2.1. Descripción del juego y reglas

Conecta 4 es un juego adversarial para dos jugadores que se desarrolla sobre un tablero vertical de seis filas por siete columnas. Cada jugador dispone de un tipo de ficha: el Humano utiliza X y la Máquina O. En cada turno se selecciona una columna y la ficha cae por gravedad hasta ocupar la posición libre más baja.

El objetivo es formar una secuencia de cuatro fichas consecutivas en dirección horizontal, vertical o diagonal. Si el tablero se llena sin que ninguno de los jugadores consiga una alineación de este tipo, la partida termina en empate.

Al tratarse de un juego de información perfecta y de suma cero, Conecta 4 puede modelarse mediante el algoritmo Minimax. Sin embargo, el elevado

número de estados posibles hace inviable explorar todo el árbol de búsqueda, por lo que el agente implementado limita la exploración a una profundidad de seis niveles y utiliza una función heurística para evaluar los estados no terminales.

Aunque Conecta 4 es un juego resuelto, ya que se ha demostrado que el primer jugador puede forzar la victoria si ambos juegan de forma óptima (Kang et al., 2019), alcanzar esa solución requiere explorar un árbol de búsqueda demasiado grande para los objetivos de este trabajo. Por ello, se emplea Minimax con poda Alfa-Beta y una evaluación heurística que permite tomar decisiones sin analizar la partida completa.

2.2. Estados y jugadas legales

- **Estado:** Corresponde a una configuración del tablero de 6×7 , determinada por la distribución de fichas X, O y casillas vacías, además del jugador al que le corresponde el turno.
- **Estado inicial:** El tablero completamente vacío, con el turno asignado al jugador Humano.
- **Jugadas legales:** En cada turno, las jugadas legales son las columnas que aún tienen al menos una casilla vacía (es decir, cuya casilla superior no está ocupada). Al elegir una columna, la ficha del jugador en turno cae hasta la posición libre más baja de dicha columna. No es posible jugar en una columna completamente llena.
- **Estados terminales:** Se alcanzan cuando un jugador consigue alinear cuatro fichas consecutivas (horizontal, vertical o diagonal) o cuando el tablero queda completamente lleno sin que exista un ganador.

2.3. Función terminal y utilidad

La partida finaliza al alcanzar cualquiera de los estados terminales descritos anteriormente. Desde la perspectiva de la Máquina (jugador maximizador), la función de utilidad asigna los siguientes valores:

- **Victoria de la Máquina:** +10,000,000, ajustado restando el número de jugadas ya realizadas para llegar a esa victoria, de modo que el algoritmo prefiera ganar en el menor número de jugadas posible.
- **Victoria del Humano:** -10,000,000, ajustado sumando ese mismo número de jugadas ya realizadas, de modo que el algoritmo prefiera retrasar una derrota inevitable.
- **Empate:** 0

El ajuste por profundidad permite diferenciar entre varias victorias o derrotas equivalentes. De esta manera, el agente prioriza ganar lo antes posible y, cuando no puede evitar perder, procura prolongar la partida.

2.4. Función de evaluación heurística

La búsqueda se limita a una profundidad de seis niveles. Si en ese punto aún no se alcanza un estado terminal, el tablero se evalúa mediante una función heurística que estima qué tan favorable resulta para la máquina. Para ello, se consideran tres criterios principales:

- **Preferencia por la columna central:** Las fichas ubicadas en la columna central participan en un mayor número de posibles alineaciones, por lo que reciben una bonificación proporcional.
- **Evaluación por ventanas de cuatro casillas:** Se recorren todas las combinaciones horizontales, verticales y diagonales de cuatro posiciones consecutivas, asignando un puntaje según la cantidad de fichas propias, rivales y casillas vacías presentes en cada ventana.

Para una ventana con n fichas propias, r fichas rivales y v casillas vacías (con $n + r + v = 4$):

$$h(ventana) = \begin{cases} +100000 & \text{si } n = 4 \\ +100 & \text{si } n = 3 \text{ y } v = 1 \\ +10 & \text{si } n = 2 \text{ y } v = 2 \\ -120 & \text{si } r = 3 \text{ y } v = 1 \\ -8 & \text{si } r = 2 \text{ y } v = 2 \\ 0 & \text{en otro caso} \end{cases}$$

Donde:

- n : número de fichas de la Máquina dentro de la ventana.
 - r : número de fichas del Humano dentro de la ventana.
 - v : número de casillas vacías dentro de la ventana.
- **Bloqueo de amenazas:** Los casos con $r = 3$ y $v = 1$ tienen una penalización notablemente mayor (-120) que su equivalente ofensivo ($+100$ para $n = 3$, $v = 1$), de modo que el agente prioriza bloquear una amenaza inminente del rival por encima de avanzar su propia ofensiva cuando ambas opciones compiten por la misma jugada.

El valor heurístico del tablero se obtiene sumando la bonificación por la columna central y los puntajes de todas las ventanas evaluadas.

$$H(tablero) = 6 * (\text{fichas propias en columna central}) + \sum_{ventana \in Ventanas} h(ventana)$$

Esta estrategia presenta una limitación conocida. Como cada ventana se analiza de forma independiente, no es posible identificar explícitamente situaciones de doble amenaza, en las que el rival genera dos oportunidades simultáneas de victoria. En estos casos, la detección depende de la profundidad alcanzada por Minimax. Como trabajo futuro, esta limitación podría reducirse incorporando un mecanismo específico para reconocer dobles amenazas o aumentando la profundidad de búsqueda.

3. Diseño de la solución y algoritmos

3.1. Minimax

El algoritmo Minimax se implementó de forma recursiva. En cada llamada se exploran las jugadas legales disponibles, se evalúa el estado resultante y, una vez obtenida la evaluación, la jugada se deshace para continuar con la siguiente alternativa. Gracias a este proceso de backtracking, el estado original del tablero se conserva durante toda la búsqueda.

La función alterna entre dos tipos de nodos:

- **Nodo maximizador (turno de la máquina):** Evalúa todas las columnas válidas y selecciona el mayor valor obtenido por sus nodos hijos.
- **Nodo minimizador (turno del humano):** Sigue el mismo procedimiento, pero elige el menor valor, ya que un beneficio para la Máquina representa una desventaja para el Humano en un juego de suma cero.

La búsqueda finaliza cuando ocurre alguna de las siguientes condiciones:

1. Se alcanza un estado terminal, ya sea por victoria de alguno de los jugadores o por empate. En este caso se devuelve el valor de utilidad definido en la sección 2.3.
2. Se alcanza la profundidad límite sin llegar a un estado terminal. En ese caso se utiliza la función heurística descrita en la sección 2.4 para estimar el valor del tablero.

Finalmente, la Máquina selecciona la columna asociada al mayor valor propagado hasta la raíz del árbol de búsqueda.

El algoritmo asume que el oponente juega de manera óptima y siempre elegirá la alternativa que minimice la utilidad de la Máquina. Bajo esta premisa, Minimax garantiza la mejor decisión posible frente al peor escenario.

3.2. Poda Alfa-Beta

Sobre la implementación de Minimax se incorporó la poda Alfa-Beta mediante dos variables de control:

- **Alfa:** representa el mejor valor que el jugador maximizador puede garantizar hasta el momento.
- **Beta:** representa el mejor valor que el jugador minimizador puede garantizar.

Durante la exploración del árbol de búsqueda:

- En cada nodo maximizador, después de evaluar un hijo, **alfa** se actualiza con el mayor valor entre el actual y el obtenido.
- En cada nodo minimizador, después de evaluar un hijo, **beta** se actualiza con el menor valor entre el actual y el obtenido.
- Si en algún momento **alfa** $\geq$ **beta**, se detiene la exploración de las columnas restantes de ese nodo, ya que ninguna de ellas puede modificar la decisión que tomará el jugador contrario en un nivel superior del árbol.

La poda Alfa-Beta reduce el número de estados evaluados sin alterar el resultado obtenido por Minimax. Las ramas descartadas no se exploran porque, de acuerdo con los valores de alfa y beta, no pueden ofrecer una alternativa mejor que la ya encontrada. En consecuencia, ambos algoritmos producen el mismo valor minimax en la raíz del árbol de búsqueda.

Para comprobar este comportamiento, el programa incorpora una segunda implementación de Minimax sin poda. En cada turno de la Máquina, ambas versiones se ejecutan sobre el mismo estado del tablero. Además de registrar el número de nodos evaluados por cada algoritmo, se compara la columna seleccionada en ambos casos. Esto permite verificar que la poda conserva la misma decisión final y, al mismo tiempo, medir la reducción de nodos obtenida.

La efectividad de la poda Alfa-Beta depende del orden en que se exploran las jugadas. En el caso ideal, cuando las mejores alternativas se evalúan primero, la complejidad de la búsqueda puede reducirse de $O(b^d)$ a aproximadamente $O(b^{d/2})$, donde (b) representa el factor de ramificación (número de columnas disponibles) y (d) la profundidad de búsqueda. Sin embargo, esta mejora no siempre se alcanza, ya que depende del orden de exploración utilizado.

En la implementación desarrollada, las columnas se recorren en un orden fijo, de la primera a la última. Como se muestra en la sección 5.3, esta decisión hace que la cantidad de nodos podados varíe entre distintas jugadas, debido a que las mejores alternativas no siempre se evalúan en primer lugar.

3.3. Modalidad de juego

El programa implementa la modalidad Humano contra Máquina. El jugador Humano (ficha X) realiza sus movimientos ingresando por consola el número de columna (1 a 7), y el programa verifica que la columna exista y no esté llena antes de aceptarla.

Una vez validada la entrada, la Máquina (ficha O) calcula su movimiento utilizando Minimax con poda Alfa-Beta y una profundidad de búsqueda de seis niveles. No se implementó la modalidad Máquina contra Máquina.

4. Implementación

4.1. Estructura del código

El programa se implementó en un único archivo, Conecta4.java, que concentra tanto el estado del juego como la lógica necesaria para su funcionamiento. La interacción se realiza mediante consola, donde el tablero se representa utilizando los caracteres X, O y . para indicar las casillas vacías.

A nivel de clase se definen las constantes principales (FILAS = 6, COLUMNAS = 7 y PROFUNDIDAD_LIMITE = 6), además de los contadores globales `nodosConPoda` y `nodosSinPoda`, utilizados para comparar el rendimiento de ambos algoritmos.

El código se organiza en cinco bloques funcionales:

- **Utilidades de tablero:** Incluyen los métodos `imprimirTablero()`, `columnaValida()`, `soltarFicha()`, `quitarFicha()`, `tableroLleno()` y `hayGanador()`, encargados de representar el tablero y aplicar las reglas del juego.
- **Función de evaluación heurística:** Implementada mediante los métodos `evaluarVentana()` y `evaluarTablero()`, responsables de asignar un puntaje a cada estado no terminal.
- **Minimax con poda Alfa-Beta:** Implementado en `minimaxAlfaBeta()`, usado para decidir la jugada real de la Máquina.
- **Minimax sin poda:** Implementado en `minimaxSinPoda()`, empleado únicamente con fines comparativos para medir la cantidad de nodos explorados.
- **Bucle principal:** Los métodos `elegirJugadaMaquina()` y `jugar()`, coordinan los turnos, ejecutan ambos algoritmos y muestran los resultados por consola.

En cada turno de la Máquina, el método `elegirJugadaMaquina()` reinicia los contadores de nodos y ejecuta tanto la versión con poda como la versión sin poda sobre el mismo estado del tablero. Posteriormente, compara la columna seleccionada por cada algoritmo y registra el número de nodos evaluados, información que se utiliza en el análisis presentado en la sección 5. Finalmente, el método `main()` crea una instancia de la clase e inicia la ejecución del juego mediante la llamada a `jugar()`.

4.2. Fragmentos comentados

```
// Verificación de 4 en línea horizontal (una de las cuatro direcciones
// evaluadas)
boolean hayGanador(char[][] t, char ficha) {
    for (int f = 0; f < FILAS; f++)
        for (int c = 0; c <= COLUMNAS - 4; c++)
            if (t[f][c] == ficha && t[f][c+1] == ficha && t[f][c+2] == ficha &&
t[f][c+3] == ficha)
                return true;
// Vertical y ambas diagonales se verifican de forma análoga
    return false;
}
```

Figura 1. Verificación de la condición de victoria (dirección horizontal, análoga para las demás).

```
// Casos base de Minimax: valores de utilidad ajustados por profundidad
if (ganaMaquina)
    return 10_000_000 - (PROFUNDIDAD_LIMITE - profundidad);
if (ganaHumano)
    return -10_000_000 + (PROFUNDIDAD_LIMITE - profundidad);
if (lleno)
    return 0;
if (profundidad == 0)
    return evaluarTablero(t, MAQUINA);
```

Figura 2. Valores de utilidad para victoria, derrota, empate y evaluación heurística al límite de profundidad.

```
// Evaluación heurística de una ventana de 4 casillas
if (contFicha == 4) puntaje += 100000;
else if (contFicha == 3 && contVacio == 1) puntaje += 100;
else if (contFicha == 2 && contVacio == 2) puntaje += 10;

// Bloqueo de amenazas: penaliza fuerte si el rival tiene 3 en línea
if (contRival == 3 && contVacio == 1) puntaje -= 120;
else if (contRival == 2 && contVacio == 2) puntaje -= 8;
```

Figura 3. Evaluación heurística de ventanas y bloqueo de amenazas del rival.

```

// Poda Alfa-Beta aplicada en el nodo maximizador (turno de la Maquina),
// alfa: mejor valor garantizado para el maximizador hasta el momento
// beta: mejor valor garantizado para el minimizador hasta el momento

if (esMaximizador) {
    int mejor = Integer.MIN_VALUE;
    for (int c = 0; c < COLUMNAS; c++) {
        if (!columnaValida(t, c)) continue;
        int fila = soltarFicha(t, c, MAQUINA);
        int valor = minimaxAlfaBeta(t, profundidad - 1, alfa, beta, false);
        quitarFicha(t, c, fila); // backtracking: se deshace la jugada
        mejor = Math.max(mejor, valor);
        alfa = Math.max(alfa, valor);
        if (alfa >= beta) break; // poda beta: se descarta el resto de columnas
    }
    return mejor;
}

```

Figura 4. Implementación de la poda Alfa-Beta en el nodo maximizador.

```

// Coloca y deshace una ficha, base del recorrido del arbol de busqueda

int soltarFicha(char[][] t, int col, char ficha) {
    for (int f = FILAS - 1; f >= 0; f--) {
        if (t[f][col] == VACIO) { t[f][col] = ficha; return f; }
    }
    return -1;
}

void quitarFicha(char[][] t, int col, int fila) {
    t[fila][col] = VACIO;
}

```

Figura 5. Mecanismo de backtracking: colocación y reversión de una ficha durante el recorrido del árbol de búsqueda.

5. Resultados y análisis

5.1. Partidas de prueba

Se ejecutaron tres partidas de prueba completas entre el jugador Humano y la Máquina, todas finalizadas con victoria de la Máquina:

- **Partida 1:** el Humano alternó jugadas entre columnas centrales y laterales (4, 4, 3, 5, 3, 6, 2, 1). La Máquina bloqueó una amenaza horizontal del Humano antes de completar, en su octava jugada, una alineación horizontal propia en la fila inferior (columnas 3 a 6).

```
Elige columna (1-7): 1
```

```
| . | . | . | . | . | . | . |
| . | . | . | . | . | . | . |
| . | . | X | O | . | . | . |
| . | . | O | X | . | . | . |
| . | . | X | O | O | O | . |
| X | X | O | X | X | X | O |
  1  2  3  4  5  6  7
```

```
Verificacion: coincide la jugada CON poda (columna 5) y SIN poda (columna 5).
```

```
La maquina juega en la columna 5
```

```
Nodos evaluados CON poda Alfa-Beta: 2921
```

```
Nodos evaluados SIN poda (Minimax puro): 53019
```

```
Tiempo de calculo: 28 ms
```

```
| . | . | . | . | . | . | . |
| . | . | . | . | . | . | . |
| . | . | X | O | . | . | . |
| . | . | O | X | O | . | . |
| . | . | X | O | O | O | . |
| X | X | O | X | X | X | O |
  1  2  3  4  5  6  7
```

```
¡Gana la MAQUINA!
```

Figura 6. Tablero final de la Partida 1

- **Partida 2:** el Humano concentró sus jugadas en las columnas centrales (4, 5, 4, 3, 4, 3, 3, 3), generando una alta ocupación alrededor de la columna 4. La Máquina respondió construyendo una alineación horizontal y obtuvo la victoria en su séptima jugada.

Elige columna (1-7): 3

	.		.		.		.		.		.	
	.		.		X		O		.		.	
	.		.		X		X		.		.	
	.		.		O		O		O		.	
	.		O		X		X		X		O	
1	2	3	4	5	6	7						

Verificacion: coincide la jugada CON poda (columna 2) y SIN poda (columna 2).

La maquina juega en la columna 2

Nodos evaluados CON poda Alfa-Beta: 1551

Nodos evaluados SIN poda (Minimax puro): 42761

Tiempo de calculo: 23 ms

	.		.		.		.		.		.	
	.		.		.		X		.		.	
	.		.		X		O		.		.	
	.		.		X		X		.		.	
	.		O		O		O		O		.	
	.		O		X		X		X		O	
1	2	3	4	5	6	7						

¡Gana la MAQUINA!

Figura 7. Tablero final de la Partida 2

- **Partida 3:** el Humano apiló repetidamente fichas en la columna central (3, 3, 3, 3, 4, 4, 4), buscando forzar una amenaza vertical. La Máquina bloqueó parcialmente la columna central y, en una posición con una sola columna disponible, completó su alineación ganadora en la octava jugada.

Elige columna (1-7): 4

	.		.		X		X		.		.		.	
	.		.		X		X		.		.		.	
	.		.		X		O		.		.		.	
	.		.		O		X		.		.		.	
	.		O		X		O		.		.		.	
	.		O		X		O		O		.		.	
1	2	3	4	5	6	7								

Verificacion: coincide la jugada CON poda (columna 1) y SIN poda (columna 1).

La maquina juega en la columna 1

Nodos evaluados CON poda Alfa-Beta: 196

Nodos evaluados SIN poda (Minimax puro): 7858

Tiempo de calculo: 4 ms

	.		.		X		X		.		.		.	
	.		.		X		X		.		.		.	
	.		.		X		O		.		.		.	
	.		.		O		X		.		.		.	
	.		O		X		O		.		.		.	
	O		O		X		O		O		.		.	
1	2	3	4	5	6	7								

¡Gana la MAQUINA!

Figura 8. Tablero final de la Partida 3

En las tres partidas, el programa confirmó explícitamente mediante el mensaje "Verificación: coincide la jugada CON poda y SIN poda" que la columna elegida por `minimaxAlfaBeta()` fue idéntica, en cada uno de los 23 turnos de la Máquina registrados, a la que habría elegido `minimaxSinPoda()` sobre el mismo estado del tablero. Esto constituye evidencia empírica directa, jugada por jugada, de que la poda no altera la decisión final del algoritmo.

5.2. Eficiencia de la poda

Tabla 1. Nodos evaluados con y sin poda Alfa-Beta por jugada

<i>Partida</i>	<i>Jugada N.º</i>	<i>Nodos sin poda</i>	<i>Nodos con poda</i>	<i>Reducción</i>
1	1	137,255	20,759	84.9%
1	2	136,647	19,876	85.5%
1	3	136,609	12,692	90.7%
1	4	128,281	17,410	86.4%
1	5	117,987	25,675	78.2%
1	6	93,491	52,981	43.3%
1	7	91,941	2,587	97.2%
1	8	53,019	2,921	94.5%
2	1	137,255	20,759	84.9%
2	2	126,927	23,207	81.7%
2	3	132,615	13,448	89.9%
2	4	133,143	14,056	89.4%
2	5	106,377	9,533	91.0%
2	6	90,686	7,241	92.0%
2	7	42,761	1,551	96.4%
3	1	137,255	12,456	90.9%
3	2	133,738	12,925	90.3%
3	3	130,436	14,103	89.2%
3	4	108,443	10,818	90.0%
3	5	51,932	7,882	84.8%
3	6	46,898	11,730	75.0%
3	7	31,307	4,156	86.7%
3	8	7,858	196	97.5%
Total	23 jugadas	2,312,861	318,962	86.2%

Los resultados muestran una reducción significativa en la cantidad de nodos explorados al utilizar poda Alfa-Beta. En las 23 jugadas analizadas, Minimax sin poda evaluó un total de 2 312 861 nodos, mientras que la versión con poda exploró 318 962, lo que representa una reducción promedio del 86.2 %.

5.3. Análisis de resultados

Los datos confirman que la poda Alfa-Beta reduce de forma sustancial el número de nodos evaluados sin sacrificar la calidad de la decisión: en las tres partidas, cada jugada elegida por `minimaxAlfaBeta()` fue exactamente la misma que habría producido `minimaxSinPoda()` sobre el mismo estado, según lo verificó explícitamente el propio programa en cada turno. La reducción promedio observada, considerando las 23 jugadas registradas, fue de 86.2%, con un mínimo de 43.3% (jugada 6 de la Partida 1) y un máximo de 97.5% (jugada 8 de la Partida 3). Esto es consistente con estudios que comparan la eficiencia de Minimax optimizado frente a otras técnicas de búsqueda en Conecta 4 (Sheoran et al., 2022).

Se observa que la reducción tiende a ser mayor conforme el tablero avanza y quedan menos columnas jugables: la jugada 8 de la Partida 3, ejecutada con una sola columna disponible, evaluó apenas 196 nodos frente a 7,858 sin poda (97.5% de reducción), ya que casi no existen ramas alternativas que podar. En contraste, en posiciones intermedias con muchas columnas abiertas y sin amenazas claras (como la jugada 6 de la Partida 1, con una reducción de solo 43.3%) la poda es menos efectiva, porque el orden fijo de exploración de columnas (de la 0 a la 6) no siempre coloca primero la mejor alternativa, retrasando las condiciones de corte $\alpha \geq \beta$. Esto es consistente con lo señalado en la sección 3.2: la efectividad de la poda depende del orden de exploración, y no está garantizada en el peor caso.

En cuanto al tiempo de ejecución, este se mantuvo entre 4 ms y 211 ms por jugada en las tres partidas, lo cual resulta imperceptible para un jugador humano y confirma que la profundidad de 6 jugadas es adecuada para mantener la interactividad del programa sin sacrificar la fuerza de juego del agente.

6. Conclusiones

El trabajo permitió implementar un agente de Conecta 4 funcional en Java, capaz de jugar contra un humano con Minimax y poda Alfa-Beta a profundidad 6, apoyado en una heurística que combina centrado de columna, evaluación de ventanas y bloqueo activo de amenazas.

La comparación jugada a jugada entre las versiones con y sin poda mostró, en la práctica, algo que la teoría ya anticipa: podar ramas no cambia la respuesta final del algoritmo, solo el camino para llegar a ella. Confirmar esto empíricamente (turno a turno, no solo en la teoría) fue uno de los aportes centrales de este trabajo.

En cuanto a eficiencia, evitar evaluar el 89.2% de los nodos explorados por Minimax puro deja clara la magnitud del ahorro que aporta la poda en un juego con un árbol de decisión tan amplio como Conecta 4, donde recorrerlo sin optimizaciones sería impráctico a mayor profundidad.

También se identificaron dos límites concretos del diseño actual. Por un lado, la heurística evalúa cada ventana de manera independiente y no reconoce posiciones de doble amenaza, algo que solo el propio Minimax puede eventualmente detectar dentro de su horizonte de búsqueda. Por otro lado, el orden fijo de exploración de columnas deja sobre la mesa parte del potencial de la poda, como se observó en las jugadas con menor reducción de nodos. Ambos puntos son mejoras naturales para una siguiente iteración del proyecto: una detección explícita de dobles amenazas, y un ordenamiento de jugadas que priorice la columna central.

7. Anexo de Prompts

Nº	Objetivo	Herramienta	Prompt	Resultado/Uso	Validación/Ajuste
P-01	Definir los parámetros de diseño del agente antes de programar (profundidad, modalidad, tipo de heurística)	Gemini 3.5 Pro	"¿Qué profundidad límite conviene para Conecta 4 en Java con Minimax y poda Alfa-Beta, de forma que el agente sea fuerte pero responda en menos de 1 segundo? ¿Qué modalidad y tipo de heurística debería usar?"	Se definieron profundidad 6, modalidad Humano vs Máquina y heurística de centrado + ventanas + bloqueo, antes de escribir código.	Se confirmaron tiempos de 4–211 ms por jugada en las partidas reales.
P-02	Evaluar si copiar el tablero en cada llamada de Minimax era eficiente y proponer una alternativa	Claude Sonnet	"Minimax copia el tablero completo en cada llamada recursiva. ¿Es la forma más eficiente, o hay una alternativa mejor?"	Se adoptó backtracking (aplicar la jugada y revertirla) en minimaxAlfaBeta(), minimaxSinPoda() y elegirJugadaMaquina().	Se comparó el número de nodos y la jugada elegida antes/después del cambio sobre la misma partida: el comportamiento no varió, solo mejoró la eficiencia.
P-03	Diseñar casos de prueba puntuales para validar la heurística (bloqueo, prioridad de victoria, doble amenaza)	Claude Sonnet	"¿Cómo se evalúa si una heurística de Conecta 4 es correcta? Necesito casos puntuales (bloqueo, prioridad de victoria, doble amenaza) antes de jugar partidas completas."	Se definieron tres escenarios con el comportamiento esperado de un jugador experto, incluida la limitación conocida ante dobles amenazas.	Se usaron como guía para diseñar las partidas reales de la sección 5.1.

8. Referencias

- Kang, X. , Wang, Y. and Hu, Y. (2019) Research on Different Heuristics for Minimax Algorithm Insight from Connect-4 Game. *Journal of Intelligent Learning Systems and Applications*, 11, 15-31. doi: 10.4236/jilsa.2019.112002.
- Nasa, R., Didwania, R., Maji, S., & Kumar, V. (2018). Alpha-Beta pruning in Mini-Max algorithm: An optimized approach for a Connect-4 game. *International Research Journal of Engineering and Technology*, 5(4), 1637–1641.
- Russell, S., & Norvig, P. (2022). *Artificial Intelligence: A Modern Approach* (4.^a ed.). Pearson.
- San, B.L., Xin, Y.J., Salam, Z.A., & Dhanoa, C.K. (2026). Connect-4 using Alpha-Beta pruning with minimax algorithm. *Journal of Applied Technology and Innovation*.
- Sheoran, K., et al. (2022). Solving Connect 4 Using Optimized Minimax and Monte Carlo Tree Search. *Advances and Applications in Mathematical Sciences*, 21(6), 3303–3313.